CRÉDITO VECINO, S. A.

Arquitectura, estrategia de RAG y seguridad

Sistema de Gestión de Microcrédito
Decisiones de diseño

Grupo 1

Integrante	Carné
Fidian Bianchi Morales Pastrán	7690-22-22593
Tedi Adolfo Castellanos Amézquita	7690-23-3016
Javier Isaac Sandoval Pérez	7690-23-5643
Cristian Roméo García de La Rosa	7690-23-12903

Guatemala, 3 de noviembre de 2026

Documento técnico breve basado en la implementación y la documentación del repositorio.

01 · ARQUITECTURA

Un núcleo financiero compartido

Crédito Vecino gestiona clientes, solicitudes, desembolsos, pagos, mora, cartera y cierres. Su asistente gerencial combina consultas a las operaciones reales con recuperación de políticas documentales. La interfaz y el chat reutilizan el mismo núcleo para mantener cifras consistentes.

Organización del sistema

Se adopta un **monolito modular con orientación hexagonal**: las reglas del dominio se separan de la presentación, el transporte HTTP y la persistencia. El dominio en TypeScript concentra amortización, dinero, mora, prelación de pagos y cartera; los adaptadores conectan estas reglas con la infraestructura.

Componente	Responsabilidad
React + Vite	Pantallas de asesor y gerencia; muestra datos, comprobantes y propuestas que requieren aprobación.
API Fastify y contratos Zod	Valida entradas, autentica operadores y expone casos de uso por HTTP.
AdaptadorNucleo y servicios	Reconstruyen el estado a una fecha de corte, coordinan operaciones y acceden a los datos persistidos.
Dominio financiero	Aplica reglas deterministas y políticas parametrizadas, independientes del modelo de lenguaje.
PGlite + pgvector	Persisten operaciones, políticas, documentos, fragmentos y vectores. PGlite ejecuta PostgreSQL embebido.
Orquestador + MCP + Ollama	Interpretan consultas, invocan herramientas permitidas y redactan respuestas con datos del núcleo o del corpus.

Flujo de una consulta gerencial

React → API /api/chat → Orquestador → herramientas MCP
Datos financieros: adaptador → dominio y persistencia.
Políticas: buscador RAG → pgvector → fragmentos citables.
Ollama interpreta y redacta; la API devuelve texto, fuentes y tarjetas de datos.

El despliegue local usa un proceso de aplicación para abrir la base PGlite. Los modelos se sirven mediante Ollama: **qwen3.5:9b** para conversación y **bge-m3** para embeddings, ambos configurables. Tras su instalación y la ingesta, esta modalidad puede operar sin servicios de IA externos.

Evidencia: src/dominio/, src/persistencia/, src/api/, src/chat/, src/mcp/, web/src/ y docs/adr/ADR-001.md.

Respuestas sustentadas y acciones controladas

Estrategia de RAG

1. Preparación. El corpus canónico contiene siete documentos de producto, políticas, preguntas frecuentes y una nota de campo. La carga verifica la huella SHA-256 del paquete y conserva los documentos originales. La fragmentación utiliza cada sección Markdown y la introducción: el corpus documentado produce 32 fragmentos con identificador, título, sección y vigencia.

2. Indexación. bge-m3 genera embeddings locales del texto y sus metadatos. Los vectores se guardan en *fragmentos_rag* con el modelo utilizado. La ingesta reemplaza los fragmentos de ese modelo dentro de una transacción para evitar duplicados.

3. Recuperación. La pregunta se transforma en un vector y se compara por distancia coseno. Por defecto se consideran los cuatro fragmentos más cercanos y se exige similitud mínima de 0.50, configurable. Si se menciona un identificador documental existente, se incorporan sus fragmentos aunque no superen el umbral semántico.

4. Respuesta. Las consultas normativas obligan a recuperar fuentes. El asistente debe distinguir vigencia y carácter normativo; si omite la cita, el orquestador agrega una fuente recuperada. Los borradores, como NOTA-CAMPO-07, se pueden resumir, pero no presentar como reglas aprobadas. Sin evidencia pertinente se informa la falta de respaldo.

Defensa en varias capas

Riesgo	Control implementado
Instrucciones maliciosas en documentos	Saneamiento de fragmentos antes de enviarlos al modelo; contenido tratado como datos y filtro de salida para el canario de prueba.
Cifras inventadas	Cálculos mediante herramientas del núcleo; contraste de montos decimales con las fuentes del turno y tarjetas con datos estructurados.
Pago sin consentimiento	El modelo puede proponer, pero no ejecutar registrar_pago. La aprobación humana genera un token de un solo uso.
Exposición de datos personales	Las herramientas gerenciales omiten DPI y teléfono; el filtro de salida añade redacción de esos patrones.
Suplantación de perfil	Con autenticación activa, usuario y rol proceden del token de sesión; las cabeceras de perfil no sustituyen esa identidad.

Los filtros reducen riesgos conocidos; no prueban inmunidad general ante todas las variantes de inyección. El umbral de búsqueda debe reevaluarse si cambia el corpus o el modelo.

Exactitud, trazabilidad y evolución

Decisión	Justificación y consecuencia
Monolito modular y fronteras de dominio	Evita la consistencia distribuida de microservicios y facilita probar reglas aisladas. Exige respetar las dependencias entre módulos. ADR-001.
Dinero con decimal.js	Objeto de valor con moneda explícita y redondeo half-up a dos decimales. Evita errores de punto flotante; requiere conversiones controladas en los límites. ADR-002.
Fecha de corte explícita	Permite reproducir saldos y mora históricos. Las herramientas financieras reciben fecha o período; la fecha de operación completa las omisiones del usuario.
Cálculos fuera del modelo de IA	El mismo núcleo alimenta API, interfaz y MCP. El RAG explica políticas y el modelo redacta; los documentos de ejemplo no determinan deudas reales.
Idempotencia y aprobación humana	Una clave identifica el pago; reintentar la misma solicitud no debe producir otro cobro. Las propuestas del chat requieren una decisión humana explícita.
Cierres definitivos persistidos	Se conservan resultado, operador y políticas utilizadas. Repetir devuelve la primera ejecución, sin recalcular las cifras históricas. ADR-003 y evolución de provisiones y devengo.
IA y búsqueda locales	Reducen la dependencia de servicios externos y mantienen el procesamiento local. Requieren memoria, modelos instalados y tiempos de respuesta adecuados al equipo.

Acceso, memoria y límites operativos

Las contraseñas se almacenan con hash scrypt. Las sesiones usan tokens opacos con expiración de 12 horas y bloqueo temporal tras cinco intentos fallidos. Los tokens y la memoria conversacional del servidor viven en memoria y se pierden al reiniciar la API. El chat conserva hasta seis turnos previos e incorpora la herramienta consultada y sus argumentos para interpretar seguimientos.

Verificación y trazabilidad

Vitest cubre dominio, API, MCP, recuperación documental, pagos, cierres y controles del chat con modelos guionados. Las evaluaciones con Ollama se ejecutan por separado y dependen del modelo configurado. La suite existente que copia la base operativa requiere datos iniciales compatibles; una base ya utilizada puede alterar los resultados esperados.

Referencias internas: docs/adr/ADR-001.md, ADR-002.md y ADR-003.md; docs/asistente-ia.md; docs/devengo-corriente.md; src/rag/buscador.ts y fragmentacion.ts; src/chat/orquestador.ts y guardias.ts; src/api/autenticacion.ts; tests/rag.test.ts, chat.test.ts, mcp.test.ts y tests/evals/.